D1 Detailed Design

Verified Student Sublease Marketplace

Team members: Danvanth Vanganuri, Jovan Jose Asker Fredy, Mohamed Aidja

Goal: Let UC students fill or find term-length subleases safely by limiting the marketplace to verified @mail.uc.edu students, matching listings to exact co-op dates, holding discriminatory listings for review, and giving property managers an approve/reject step before move-in.

Scope: This D1 details C3 Listing Service and C5 Search & Match. C2 Identity, C1 Web Client, C4 Compliance, C6 Messaging, C7 Approval, and C8 Notification are deferred to D2.

1. Header, Scope, and Conventions

Rounded boxes are entities owned by the component named in the box title. A dashed box is a transient request object that is never stored. A solid line is a persistent foreign-key relationship. A dashed arrow is a read-only query dependency. Keys are marked in the field list: [PK] primary key, [FK] foreign key, [UQ*] partial unique index, and a trailing ? means nullable. Cardinality is written at both ends of every line as 1, 0..1, or 0..*, read from the box next to the label. Dates use ISO 8601 (yyyy-mm-dd), datetimes are UTC, and money is stored as integer cents.

2. Data Model, the D1 Diagram

C3 is the source of truth for listing lifecycle state and C5 keeps a derived search index. This follows the D0 ownership rule: C2 owns account data, C3 owns listings and outbox rows, and C5 owns the search index. A listing is copied into C5 only when it is PUBLISHED, so HELD_FOR_REVIEW and REJECTED listings never appear in search (AC-01.3).

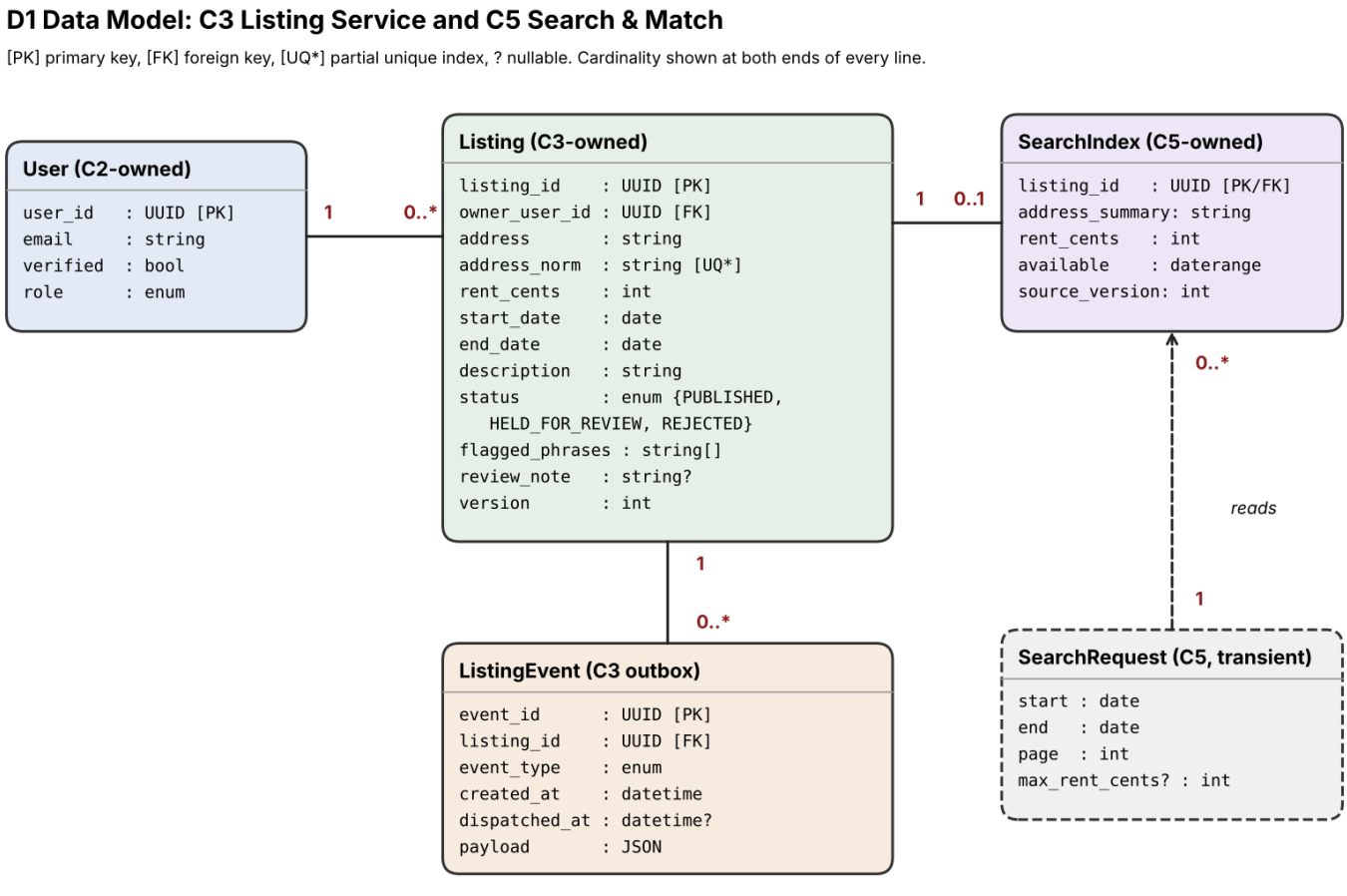


Figure 1. D1 data model for the listing and search components. Only behavior-relevant attributes are shown.

Requirement trace: US-01, US-02, US-05; AC-01.1, AC-01.3, AC-01.4; D-05, D-06.

Decision	Choice and reason	Req.
Entity vs attribute	Listing is an entity because it has its own lifecycle, owner, dates, status, and events. Address and description are attributes because they describe exactly one listing and have no identity or behavior of their own. flagged_phrases and review_note are stored on Listing so the review queue can be served without calling C4 again.	US-01, AC-01.3
1-to-many: User to Listing	One verified user publishes 0..* listings; each listing has exactly one owner. Not many-to-many because a sublease is offered by one person.	US-01
1-to-many: Listing to ListingEvent	A listing writes 0..* outbox events as its state changes (one per publish or release). Events are rows, not an attribute, so each can be dispatched, retried, and audited independently via dispatched_at.	D-05
1-to-0..1: Listing to SearchIndex	C5 keeps at most one derived row per listing, keyed by listing_id. The row exists only while the listing is PUBLISHED. This avoids scanning every C3 listing per search while C3 remains the source of truth. A separate table, not a view, so held and rejected listings cannot leak into search even through a query bug.	US-02, AC-01.3
Search availability as one range	SearchIndex stores availability as a single daterange (start and end explicit) so exact co-op coverage is one containment test, not two comparisons.	D-06
Stale-write guard	Listing.version increments on each state change and is copied to SearchIndex.source_version. C5 ignores an upsert whose version is not newer, so out-of-order event delivery cannot overwrite newer data.	D-05
Indexes	C3: partial unique index on address_norm WHERE status IN (PUBLISHED, HELD_FOR_REVIEW) to enforce the duplicate-address 409; index on (status) for the review queue; index on (owner_user_id) for the FK. ListingEvent: partial index on created_at WHERE dispatched_at IS NULL so the poller finds pending events quickly. C5: GiST index on available for coverage queries; btree on rent_cents for the optional filter.	AC-01.1, D-06
Model choice	Relational SQL (PostgreSQL), not a document store, because listing state transitions and the outbox write need one ACID transaction and the relationships are structured. D0 already selects a single modular monolith.	D-05

Status values: PUBLISHED, HELD_FOR_REVIEW, REJECTED. Event types: LISTING_PUBLISHED, LISTING_RELEASED.

3. Core Algorithms

Algorithm A: Exact date-coverage search (C5)

Part	Detail
What and why	Return only PUBLISHED listings whose availability fully covers the student's co-op term. D-06 requires full coverage, not simple overlap, so a listing that ends before the term ends must never match (US-02).
Inputs / outputs	In: start:date, end:date, page:int (1 to 50), maxRentCents:int? (1 to 500000). Out: results[] of {listingId:UUID, addressSummary:string, rentCents:int, availableStart:date, availableEnd:date}, total:int, page:int, pageSize:int (fixed at 20). Ordering: rentCents ascending, then listingId ascending, so pages are deterministic.
Method	Query SearchIndex with `available @> daterange(:start, :end, '[]')` and, if given, `rent_cents <= :max`, then ORDER BY rent_cents, listing_id with LIMIT 20 and OFFSET (page-1)*20.
Complexity at realistic size	A btree on start and a btree on end cannot both bound a two-sided condition: the planner uses one range and filters the rest, so the honest cost is O(log N + s) where s is the rows matching the first bound. A GiST index on the daterange column supports the @> containment operator directly and avoids that. Expected N is about 2,000 published listings (a campus-scale estimate); at that size even a sequential scan takes a few milliseconds. At 100x (200,000 rows) the GiST index keeps the query well under the 60-second search budget, so the 100x difference does not change the design. These are estimates to confirm with EXPLAIN ANALYZE during implementation.
Why this one	Alternatives: (1) full table scan, simplest but linear in N; (2) two btree indexes, which cannot bound both sides; (3) a hosted search engine (Elasticsearch, Algolia), which adds a service, cost, and sync problem the D0 requirements do not need. GiST on a range type is built into PostgreSQL. A separate C5 table in the same database keeps the read-optimized rows apart from the C3 write model and structurally excludes unpublished listings (AC-01.3) without adding a hosted service.
Edge cases	Empty result: 200 with results=[] (US-02). end <= start, malformed date, page < 1, page > 50, or maxRentCents <= 0: 400 (D-06). Duplicate event delivery: upsert on listing_id. Out-of-order event: ignore when source_version is not newer (D-05). Listing leaves PUBLISHED: delete its SearchIndex row (AC-01.3). Tie on rent: broken by listing_id.

Algorithm B: Listing state transition and transactional outbox (C3)

Part	Detail
What and why	Publish, hold, or review a listing without ever publishing an unscreened listing and without losing downstream events (US-01, US-05, AC-01.3, AC-01.4, D-05).

Part	Detail
Inputs / outputs	Submit in: {address:string 1-200, rentCents:int 1-500000, startDate:date, endDate:date, description:string 0-2000}, caller claims, C4 result {flagged:bool, matches:string[]}. Submit out: listingId:UUID, status in {PUBLISHED, HELD_FOR_REVIEW}, flaggedPhrases:string[]. Review in: listingId, decision in {RELEASE, REJECT}, note:string? (0-500). Review out: listingId, status in {PUBLISHED, REJECTED}. REJECTED is only reachable from review.
Method	Submit: validate dates and rent; require verified caller; call C4. If C4 flags the text or is unavailable, fail closed: write the listing as HELD_FOR_REVIEW with flagged_phrases and write no outbox row. Otherwise write the listing as PUBLISHED and a LISTING_PUBLISHED outbox row in the same SQL transaction. Review RELEASE: in one transaction, move HELD to PUBLISHED, increment version, and write a LISTING_RELEASED outbox row. Review REJECT: move HELD to REJECTED and store the note; no search event is needed because held listings are not indexed.
Outbox drain	A poller inside the Node process runs every 30 seconds. It selects up to 50 rows WHERE dispatched_at IS NULL ORDER BY created_at with FOR UPDATE SKIP LOCKED, performs the C5 upsert and the C8 notification, then sets dispatched_at. A failed delivery leaves the row undispatched and is retried on the next cycle. Worst-case delay is one interval plus retries, well inside D0's 5-minute notification budget.
Complexity	O(1) application work plus a bounded phrase scan (at most 2,000 characters against a fixed list) and a constant number of row writes per request. At 100x listings, per-request work is unchanged; only total outbox volume grows, and the partial index keeps the poller query fast.
Why this one	Calling C5 and C8 synchronously would make submission depend on their availability. A message broker adds cost and another hosted dependency. D-05 selected the transactional outbox.
Edge cases	Invalid dates or rent: 400, no row written (AC-01.1). Unverified caller: 403 (US-01). Duplicate active address: 409 from the partial unique index (AC-01.1). C4 unavailable: HELD_FOR_REVIEW (AC-01.3). Review of a non-held listing: 409 (AC-01.4). Unknown listing: 404. Crash between writes: impossible by design, the listing and outbox rows commit together (D-05).

4. Build-versus-Reuse Decisions

Piece	Build / reuse	Library or service	License	Reason and check (maturity, performance, fit)
Listing state machine + outbox rows	Build	Project code on PostgreSQL	N/A	Product-specific rules (hold, release, reject) and timing requirements belong to the team.
Outbox poller	Build	Small loop + SELECT ... FOR UPDATE SKIP LOCKED	N/A	About 40 lines; SKIP LOCKED is a built-in Postgres feature, so no queue library is needed at this volume.
Phrase scan input to C4	Build	Project code	N/A	Bounded string match against a fixed list; the phrase list itself is detailed in D2 (C4).
SQL database	Reuse	Managed PostgreSQL	PostgreSQL License	Mature (decades), ACID transactions, constraints, GiST ranges; fit for 2,000 rows with huge headroom.
Search index	Reuse (built-in PostgreSQL)	GiST index on daterange	PostgreSQL License	Built into core, no extension; avoids a second service and its cost at semester scale.
DB driver	Reuse	node-postgres (pg)	MIT	Widely used, stable, supports parameterized queries and transactions; thin enough not to hide the SQL.
HTTP API	Reuse	Node.js + Express	MIT	Mature, huge community; fits the team's JavaScript/TypeScript experience; throughput is far above need.
Type checking	Reuse	TypeScript	Apache-2.0	Catches request/response type mistakes at compile time; no runtime cost.
Request validation and date parsing	Reuse	zod	MIT	Maintained, TypeScript-native, parses and validates dates and ranges so we do not hand-roll parsing.
Token verification	Reuse	jose (JWT)	MIT	Maintained, standards-based; verifies tokens issued by C2 so C3 does not hand-roll auth.
Authentication / identity	Reuse	C2 one-time-link flow	N/A	No password storage to build; C2 already owns verification.

5. API Contract

All paths are under /api/v1. All endpoints require `Authorization: Bearer <token>`; a missing or invalid token returns 401 UNAUTHENTICATED. Malformed or out-of-range input returns 400 INVALID_INPUT on every endpoint.

Endpoint	Inputs (type, required, range)	Success response	Errors
POST /api/v1/listings	address:string, required, 1-200 chars; rentCents:int, required, 1-500000; startDate:date, required; endDate:date, required, after startDate; description:string, optional, 0-2000 chars	201 {listingId:UUID, status:PUBLISHED|HELD_FOR_REVIEW, flaggedPhrases:string[]}	400 INVALID_INPUT; 401 UNAUTHENTICATED; 403 VERIFICATION_REQUIRED (caller not verified); 409 DUPLICATE_LISTING (same normalized address already active)
GET /api/v1/listings/search	start:date, required; end:date, required, after start; maxRentCents:int, optional, 1-500000; page:int, optional, 1-50, default 1	200 {results:[{listingId:UUID, addressSummary:string, rentCents:int, availableStart:date, availableEnd:date}], total:int, page:int, pageSize:int (20)}	400 INVALID_INPUT (bad date, end <= start, page out of range); 401 UNAUTHENTICATED; 403 VERIFICATION_REQUIRED
GET /api/v1/review-queue	none; caller must have REVIEWER role	200 {listings:[{listingId:UUID, description:string, flaggedPhrases:string[]}]}	401 UNAUTHENTICATED; 403 FORBIDDEN (not a reviewer)
POST /api/v1/listings/{id}/review	id:UUID, path, required; decision:RELEASE|REJECT, required; note:string, optional, 0-500 chars	200 {listingId:UUID, status:PUBLISHED|REJECTED}	400 INVALID_INPUT; 401 UNAUTHENTICATED; 403 FORBIDDEN; 404 NOT_FOUND (unknown id); 409 NOT_HELD_FOR_REVIEW

Example request and response:

```
GET /api/v1/listings/search?start=2027-01-11&end=2027-04-30&page=1&maxRentCents=90000
Authorization: Bearer <token>

200 OK
{"results":[{"listingId":"7b2e4c1a-9d3f-4a8e-b6c5-2f1d0a9e8b73","addressSummary":"Main St",
  "rentCents":85000,"availableStart":"2027-01-01","availableEnd":"2027-06-01"}],
 "total":1,"page":1,"pageSize":20}
```

Versioning: the URL prefix /api/v1 versions the whole contract. A breaking change is removing or renaming a field, changing a field's type or meaning, adding a new required input, or changing a status or error code callers handle; breaking changes ship as /api/v2, while adding optional inputs or new response fields is non-breaking.

6. Technology Choices with Justification

Database: PostgreSQL. Team skill fit: Danvanth has used SQL Server at a co-op and Postgres via Supabase in projects. Licensing: the PostgreSQL License is permissive. Community support: decades of documentation and answers. Performance: a GiST range lookup on about 2,000 rows should complete in single-digit milliseconds (an estimate to confirm with EXPLAIN ANALYZE). Cost and hosting: managed Postgres on the Supabase free tier ($0) for the semester. We passed over MongoDB because the listing-plus-outbox write needs one multi-row ACID transaction and the date-range coverage query maps directly to a Postgres range type.

Backend: Node.js with TypeScript and Express. Team skill fit: Danvanth has built JavaScript/TypeScript web apps with Next.js and Supabase, and one language across front and back end lets all three teammates review each other's code. Licensing: Node.js and Express are MIT, TypeScript is Apache-2.0. Community support: among the largest ecosystems, with mature zod, pg, and jose libraries. Performance: a single Node process handles hundreds of requests per second, far above a campus marketplace's load. Cost and hosting: one small web service, no extra services. We passed over NestJS because its module and decorator structure adds learning overhead for a three-person semester project, and FastAPI because it would split the stack into two languages.

Front end: React. Team skill fit: Danvanth has built React and Next.js apps; React keeps the front end in the same language as the backend. Licensing: React is MIT. Community support: the largest front-end ecosystem, with accessible component libraries supporting US-04 (semantic HTML and ARIA). Performance: the client only renders results; search runs on the server. Cost and hosting: static build served from the same host or a free static tier. We passed over Angular for its heavier framework and learning curve, and over native iOS and Android clients per D0's D-08.

Messaging: PostgreSQL transactional outbox with an in-process poller (no broker). Team skill fit: the team already writes the SQL; no new system to learn. Licensing: nothing new beyond PostgreSQL. Community support: the outbox pattern and SKIP LOCKED queues are well documented. Performance: a 30-second poll against a 5-minute budget leaves a large margin. Cost and hosting: $0 extra. We passed over Amazon SQS and RabbitMQ because each adds a hosted service, credentials, and cost for a few events per day.

Hosting: one managed web service plus managed Postgres. Team skill fit: Danvanth has used Supabase for hosted Postgres, and Render deploys a Node service from a Git repository with no server administration. Licensing: the host adds no

licensing burden to our MIT/Apache dependencies. Community support: Render and Supabase have public documentation and active communities. Performance: a single region instance meets D0's 60-second search budget by a wide margin. Cost and hosting: estimated about $7 per month total (a Render web service plus the Supabase free tier; prices to be rechecked at deploy time). We passed over AWS EC2 plus RDS because of setup and operations cost for a semester project, and over eight separately hosted microservices per D0.

Team experience: Danvanth Vanganuri: SQL Server (co-op), Postgres/Supabase, Next.js/React. Jovan Jose Asker Fredy and Mohamed Aidja have both used SQL and React, from high school.

D1 boundary: C2 Identity, C1 Web Client, C4 Compliance, C6 Messaging, C7 Approval, and C8 Notification are intentionally deferred to D2.